

AWS — Production-Ready VPC + Private RDS (MySQL)

Complete, step-by-step implementation guide (console + CLI + verification + hardening + testing + cleanup). Replace placeholder values (< . . . >) with your own names/IDs.

Quick project summary / goals

- Create an isolated, production-style VPC with public and private subnets (multi-AZ ready).
- Deploy RDS (MySQL) in private subnets (no public IP).
- Deploy an EC2 bastion/app server in the public subnet to access RDS.
- Implement route tables, security groups, IAM role for secrets, backups, encryption, and cleanup.
- Deliverables: working RDS accessible only from bastion, automated backups, monitoring & alerts, secure credential handling via Secrets Manager.

Table of contents

1. Naming + placeholders (use these consistently)
2. High level architecture (diagram)
3. Pre-flight (cost control & account checks)
4. Step-by-step build (Console click-by-click + CLI snippets)
 - VPC & subnets
 - IGW, NAT, route tables
 - Security groups + optional NACLs
 - DB subnet group

- RDS instance (MySQL) — production options (Multi-AZ, encryption, monitoring)
 - IAM role & Secrets Manager (EC2 reads DB credentials)
 - EC2 bastion/app server (attach role & test)
 - Optional: Snapshot export to S3 (IAM role + export task)
5. Tests & validation checklist (exact commands/SQL)
 6. Troubleshooting common errors & fixes
 7. Cleanup steps (console + CLI)

1) Naming & placeholders (replace everywhere)

Use a consistent prefix: proj or harshitha-rds

Examples used below:

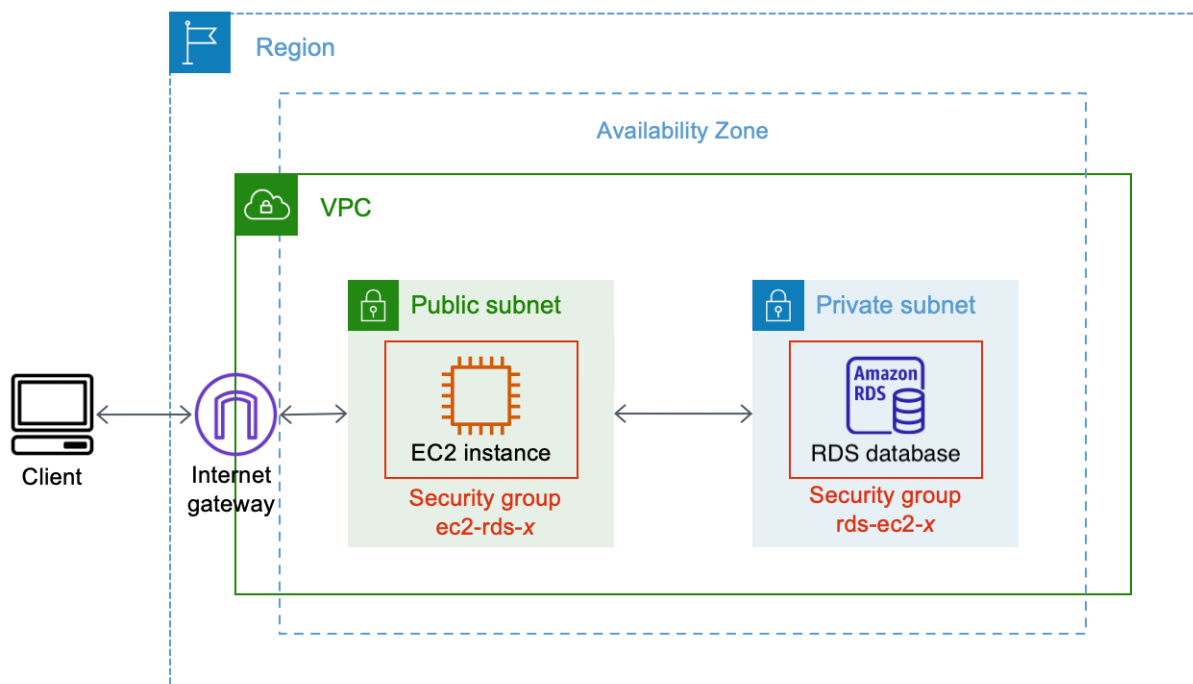
- VPC: prod-vpc
- Public subnet(s): prod-public-a (CIDR 10.0.1.0/24), prod-public-b (optional)
- Private subnets: prod-private-a (10.0.1.0/24), prod-private-b (10.0.2.0/24)
- Internet Gateway: prod-igw
- Route tables: prod-public-rt, prod-private-rt
- Security groups: sg-bastion, sg-rds
- DB subnet group: prod-db-subnet-group
- RDS instance id: prod-rds-db
- EC2 bastion: prod-bastion
- Secrets Manager secret name: prod/rds/harshi
- IAM role for EC2: EC2SecretsRole

- S3 exported snapshot bucket: prod-rds-exports-<suffix>
- Region: <REGION> (e.g., ap-south-1)
- Account id: <ACCOUNT_ID>

2) Architecture

Why this layout:

- Public subnet = bastion (only place with public IP).
- RDS in private subnets - no public endpoint.
- Internet Gateway provides internet access.
- Security groups reference each other (SG→SG) — recommended.



3) Pre-flight checks & cost control

- Use a sandbox AWS account.
- Create a budget: Billing → Budgets → create a cost budget with a low threshold (\$1–\$10) and email alert.
- Use small instance types for practice (t3.micro, db.t3.micro).

4) Full build — console steps (detailed) + CLI snippets

Work in this order. For each Console section I give click-by-click; CLI snippets follow where useful.

4.1 Create VPC

Console steps

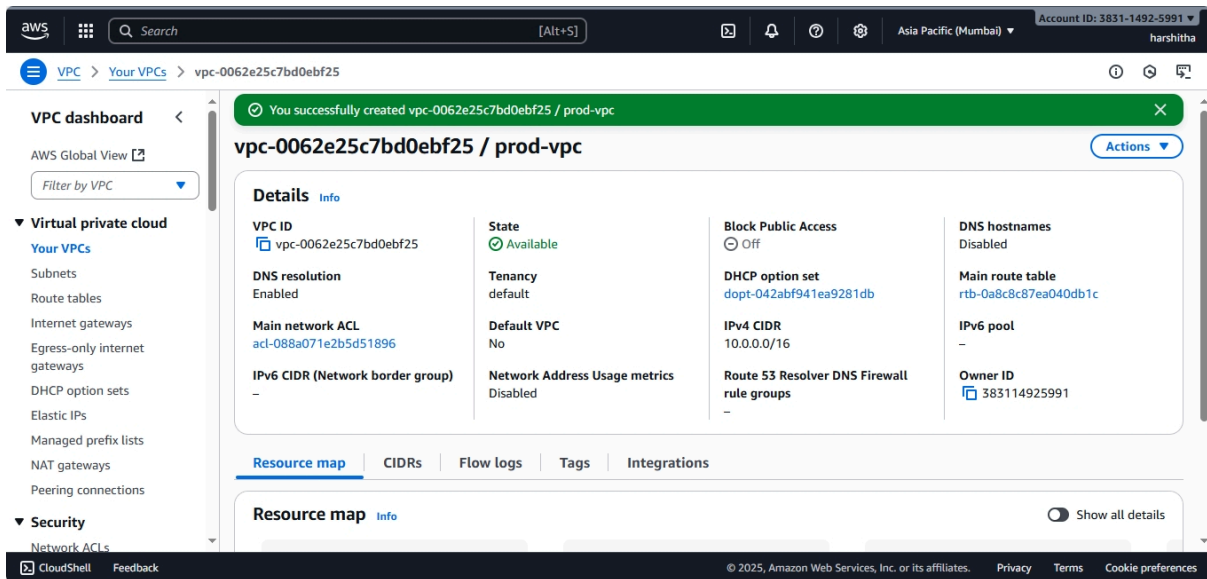
1. Services → VPC → Your VPCs → Create VPC.
2. Choose “VPC only” or “VPC and more”.
3. Name: prod-vpc. IPv4 CIDR: 10.0.0.0/16. Tenancy: default.

Create.

CLI

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16 \  
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=prod-vpc}]' \  
  --region <REGION>
```

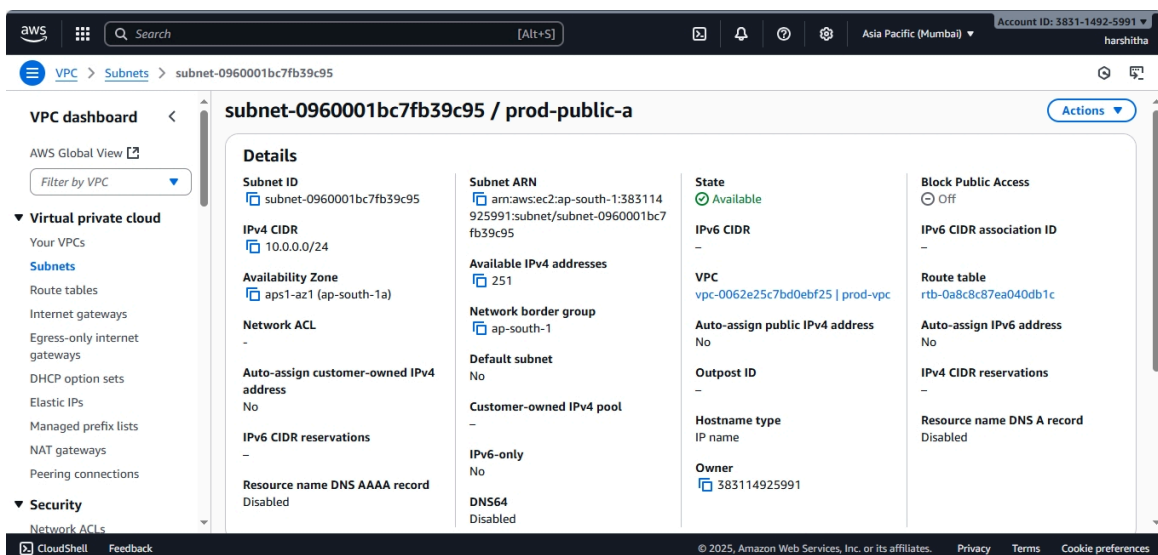
Record the returned VpcId.



4.2 Create subnets (public + private in two AZs for HA)

Console

1. VPC → Subnets → Create subnet.
 - Select VPC: prod-vpc.
 - Name: prod-public-a. AZ: choose <REGION>a. IPv4 CIDR: 10.0.1.0/24. Enable auto-assign public IPv4.
 - Create.



2. Create private subnet prod-private-a in same AZ: CIDR 10.0.1.0/24 (auto-assign disabled).

The screenshot shows the AWS Management Console interface for the subnet-04ace24b23771c9d0. The left sidebar contains the VPC dashboard and a list of VPC resources. The main content area displays the details of the subnet, including its ID, ARN, CIDR block, availability zone, and various configuration options like network ACL, auto-assign public IP, and DNS settings.

subnet-04ace24b23771c9d0 / prod-private-a			
Subnet ID subnet-04ace24b23771c9d0	Subnet ARN arn:aws:ec2:ap-south-1:383114925991:subnet/subnet-04ace24b23771c9d0	State Available	Block Public Access ☐ Off
IPv4 CIDR 10.0.1.0/24	Available IPv4 addresses 251	IPv6 CIDR -	IPv6 CIDR association ID -
Availability Zone aps1-az1 (ap-south-1a)	Network border group ap-south-1	VPC vpc-0062e25c7bd0ebf25 prod-vpc	Route table rtb-0a8c8c87ea040db1c
Network ACL -	Default subnet No	Auto-assign public IPv4 address No	Auto-assign IPv6 address No
Auto-assign customer-owned IPv4 address No	Customer-owned IPv4 pool -	Outpost ID -	IPv4 CIDR reservations -
IPv6 CIDR reservations -	IPv6-only No	Hostname type IP name	Resource name DNS A record Disabled
Resource name DNS AAAA record Disabled	DNS64 Disabled	Owner 383114925991	

3. Repeat in another AZ for HA: prod-private-b (10.0.3.0/24) and prod-public-b (optional).

The screenshot shows the AWS Management Console interface for the subnet-09fcfb7fb99a45d99. The left sidebar contains the VPC dashboard and a list of VPC resources. The main content area displays the details of the subnet, including its ID, ARN, CIDR block, availability zone, and various configuration options like network ACL, auto-assign public IP, and DNS settings.

subnet-09fcfb7fb99a45d99 / prod-private-b			
Subnet ID subnet-09fcfb7fb99a45d99	Subnet ARN arn:aws:ec2:ap-south-1:383114925991:subnet/subnet-09fcfb7fb99a45d99	State Available	Block Public Access ☐ Off
IPv4 CIDR 10.0.2.0/24	Available IPv4 addresses 251	IPv6 CIDR -	IPv6 CIDR association ID -
Availability Zone aps1-az3 (ap-south-1b)	Network border group ap-south-1	VPC vpc-0062e25c7bd0ebf25 prod-vpc	Route table rtb-0a8c8c87ea040db1c
Network ACL -	Default subnet No	Auto-assign public IPv4 address No	Auto-assign IPv6 address No
Auto-assign customer-owned IPv4 address No	Customer-owned IPv4 pool -	Outpost ID -	IPv4 CIDR reservations -
IPv6 CIDR reservations -	IPv6-only No	Hostname type IP name	Resource name DNS A record Disabled
Resource name DNS AAAA record Disabled	DNS64 Disabled	Owner 383114925991	

CLI (example)

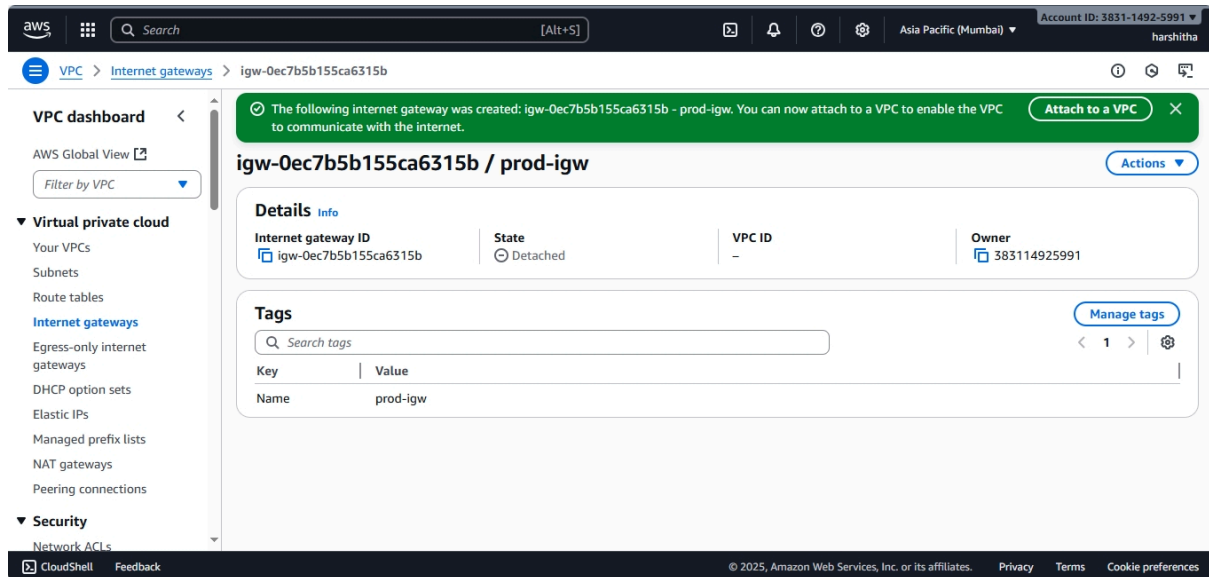
```
aws ec2 create-subnet --vpc-id <VPC_ID> --cidr-block 10.0.1.0/24 \
  --availability-zone <REGION>a --tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value=prod-public-a}]'
```

Repeat for each subnet.

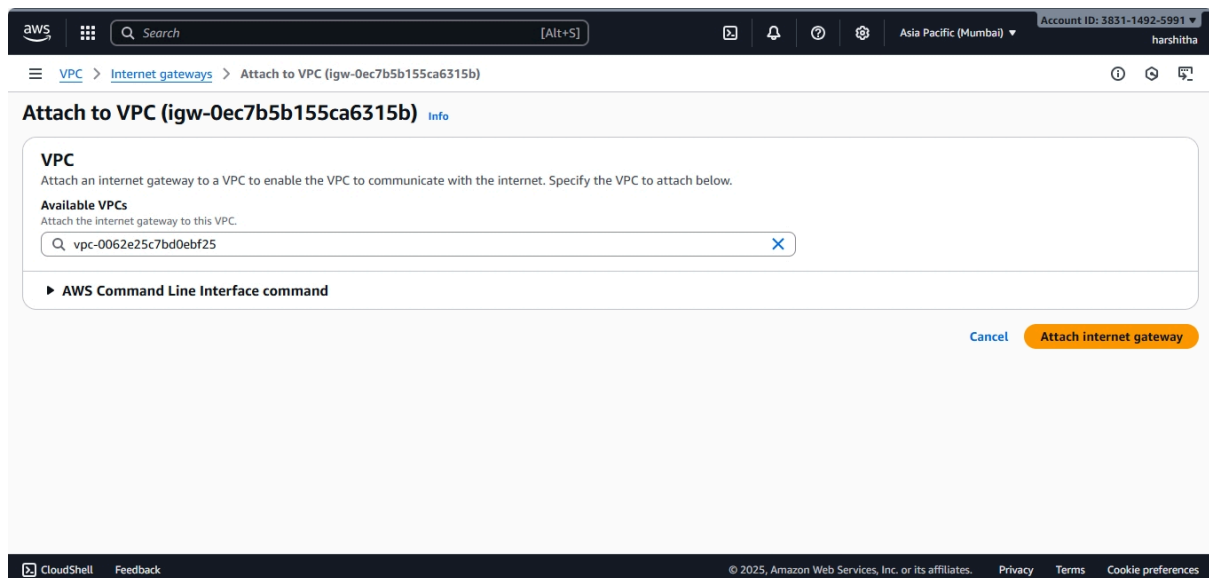
4.3 Create Internet Gateway and attach

Console

1. VPC → Internet Gateways → Create internet gateway → Name: prod-igw.

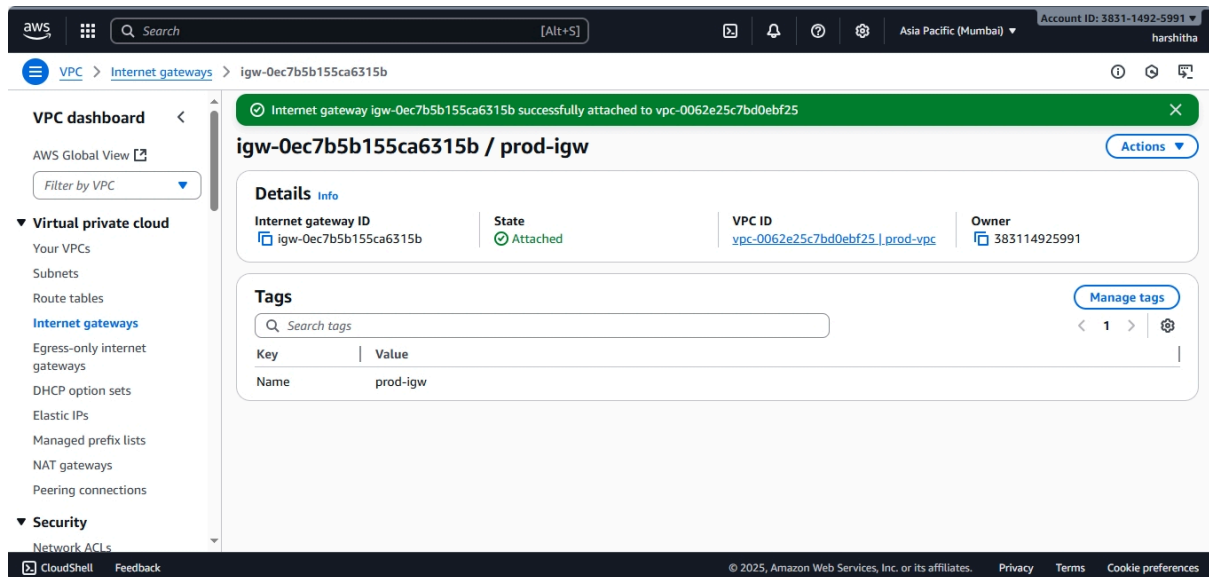


2. Select prod-igw → Actions → Attach to VPC → choose prod-vpc.



CLI

```
aws ec2 create-internet-gateway --tag-specifications  
'ResourceType=internet-gateway,Tags=[{Key=Name,Value=prod-igw}]'  
aws ec2 attach-internet-gateway --internet-gateway-id <IGW_ID> --vpc-id <VPC_ID>
```



4.4 Create NAT Gateway (public subnet) — provides outbound for private subnets(optional:if you want to allow outbound traffic to private subnet)

Console

1. VPC → Elastic IPs → Allocate Elastic IP → tag name prod-eip.
2. VPC → NAT Gateways → Create NAT gateway. Choose subnet prod-public-a, Elastic IP = prod-eip. Create and wait for Available.

Notes: NAT Gateway costs money—remove when done.

CLI (allocate EIP + create NAT)

```
aws ec2 allocate-address --domain vpc  
aws ec2 create-nat-gateway --subnet-id <PUBLIC_SUBNET_ID> --allocation-id  
<EIP_ALLOC_ID>
```


4.5 Route tables: public and private routes

Console

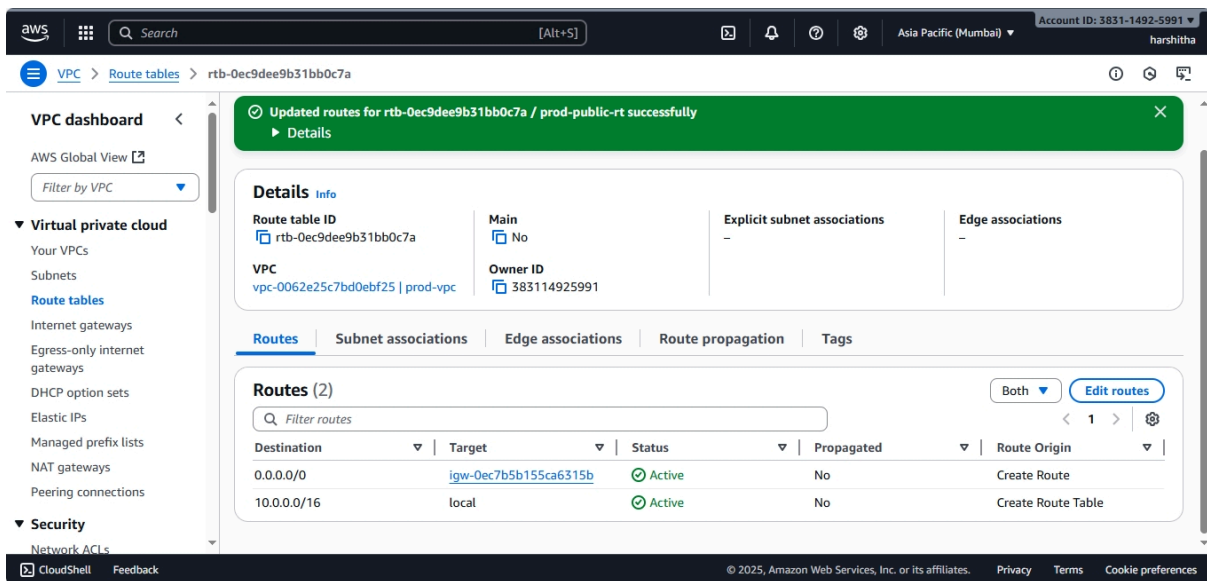
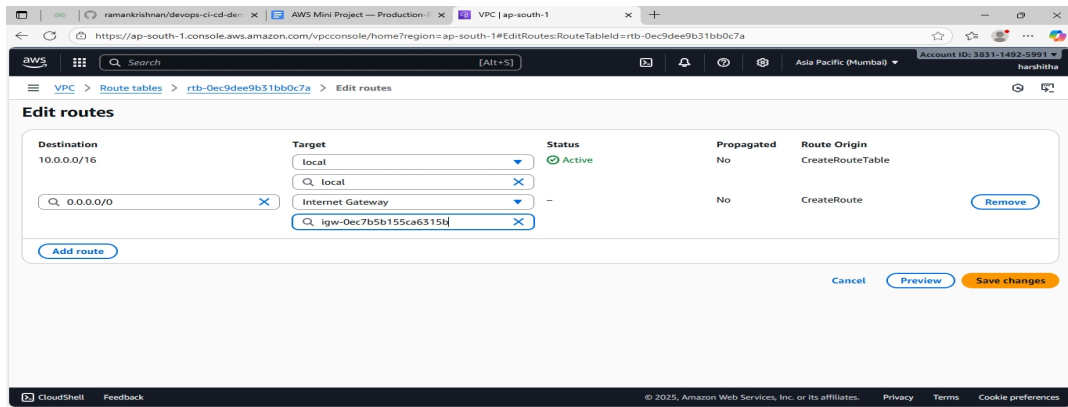
1. VPC → Route tables → Create route table → Name prod-public-rt → VPC prod-vpc.

The screenshot shows the 'Create route table' page in the AWS Management Console. The page title is 'Create route table' with an 'Info' link. Below the title is a description: 'A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.' The 'Route table settings' section includes a 'Name - optional' field with the value 'prod-public-rt' and a 'VPC' dropdown menu showing 'vpc-0062e25c7bd0ebf25 (prod-vpc)'. The 'Tags' section shows a table with one tag: 'Name' with value 'prod-public-rt'. There is an 'Add new tag' button and a note 'You can add 49 more tags.'

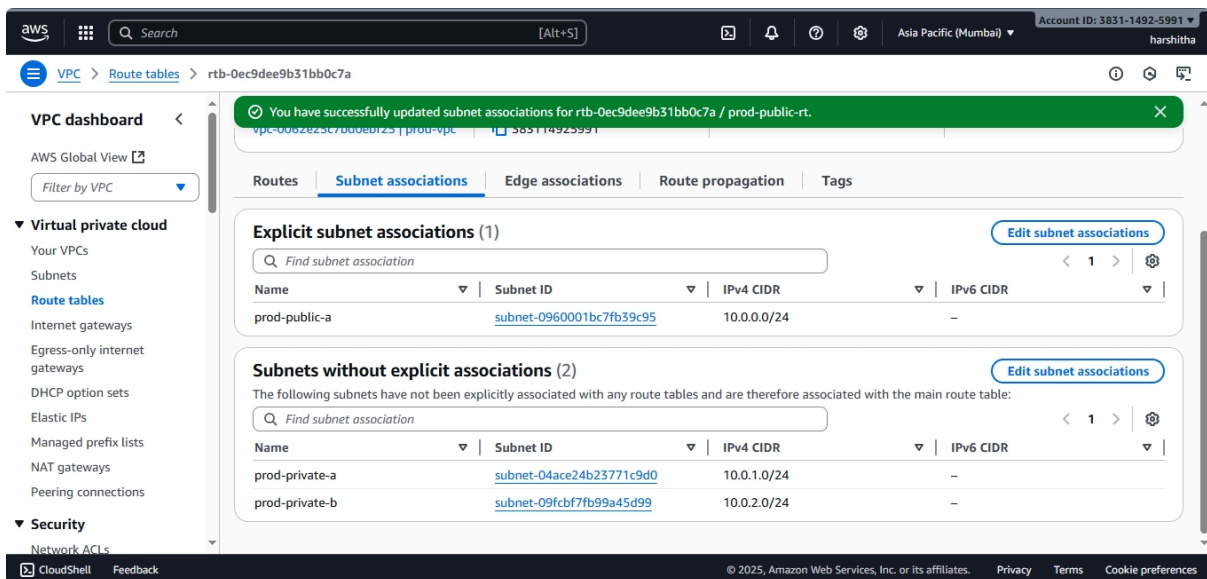
The screenshot shows the 'Route table' page for 'rtb-0ec9dee9b31bb0c7a / prod-public-rt'. A green notification bar at the top says 'Route table rtb-0ec9dee9b31bb0c7a | prod-public-rt was created successfully.' The page has a left sidebar with 'VPC dashboard' and 'Virtual private cloud' sections. The main content area shows 'Details' for the route table, including 'Route table ID', 'VPC', 'Main' status, 'Owner ID', 'Explicit subnet associations', and 'Edge associations'. Below this is a 'Routes' section with a table showing one route: '10.0.0.0/16' with target 'local', status 'Active', propagated 'No', and route origin 'Create Route Table'. There is an 'Edit routes' button.

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	Create Route Table

2. Select prod-public-rt → Routes → Edit routes → Add route 0.0.0.0/0 → Target: Internet gateway → choose prod-igw. Save.



3. Select prod-public-rt → Subnet associations → Edit subnet associations → associate prod-public-a (and prod-public-b if exists).



CLI

```
aws ec2 create-route-table --vpc-id <VPC_ID> --tag-specifications  
'ResourceType=route-table,Tags=[{Key=Name,Value=prod-public-rt}]'  
aws ec2 create-route --route-table-id <PUB_RT_ID> --destination-cidr-block 0.0.0.0/0  
--gateway-id <IGW_ID>  
aws ec2 associate-route-table --route-table-id <PUB_RT_ID> --subnet-id  
<PUBLIC_SUBNET_ID>  
# private RT -> route to NAT  
aws ec2 create-route --route-table-id <PRIV_RT_ID> --destination-cidr-block 0.0.0.0/0  
--nat-gateway-id <NAT_ID>
```

4. Create route table prod-private-rt → add route 0.0.0.0/0 → Target: NAT Gateway prod-nat. Associate with prod-private-a, prod-private-b.

Route table rtb-041092791d05e87be / prod-private-rt

Details

Route table ID rtb-041092791d05e87be	Main No	Explicit subnet associations -	Edge associations -
VPC vpc-0062e25c7bd0ebf25 prod-vpc	Owner ID 383114925991		

Routes (1)

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	Create Route Table

Route table rtb-041092791d05e87be / prod-private-rt

Details

Route table ID rtb-041092791d05e87be	Main No	Explicit subnet associations 2 subnets	Edge associations -
VPC vpc-0062e25c7bd0ebf25 prod-vpc	Owner ID 383114925991		

Explicit subnet associations (2)

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
prod-private-a	subnet-04ace24b23771c9d0	10.0.1.0/24	-
prod-private-b	subnet-09fcfb7fb99a45d99	10.0.2.0/24	-

Subnets without explicit associations (0)

The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

4.6 Create Security Groups

Security groups (SG) must follow least privilege and reference SGs where possible.

Console: create sg-bastion

1. VPC → Security Groups → Create security group
 - Name: bastion-sg
 - Description: Allow SSH from admin IP
 - VPC: prod-vpc
 - Inbound rules:
 - Type: SSH (TCP 22) → Source: My IP (enter your public IP)
 - Outbound: Allow all (default)

The screenshot shows the 'Create security group' page in the AWS Management Console. The page is titled 'Create security group' with a sub-header 'Info'. Below the title, a note states: 'A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.' The form is divided into two main sections: 'Basic details' and 'Inbound rules'.

Basic details

- Security group name**: A text input field containing 'bastion-sg'. A note below the field states: 'Name cannot be edited after creation.'
- Description**: A text input field containing 'Allow SSH from admin IP'.
- VPC**: A dropdown menu showing 'vpc-0062e25c7bd0ebf25 (prod-vpc)'.

Inbound rules

The 'Inbound rules' section has a table with columns: Type, Protocol, Port range, Source, and Description - optional. The first rule is configured as follows:

Type	Protocol	Port range	Source	Description - optional
SSH	TCP	22	My IP	

Below the table, the source IP is specified as '160.238.72.133/32'.

The screenshot shows the 'sg-0314d7087a95ad40b - bastion-sg' page in the AWS Management Console. The page is titled 'sg-0314d7087a95ad40b - bastion-sg' with a sub-header 'Details'. A green notification banner at the top states: 'Security group (sg-0314d7087a95ad40b) was created successfully'. Below the notification, the page shows the details of the security group.

Details

Property	Value
Security group name	bastion-sg
Security group ID	sg-0314d7087a95ad40b
Description	Allow SSH from admin IP
VPC ID	vpc-0062e25c7bd0ebf25
Owner	383114925991
Inbound rules count	1 Permission entry
Outbound rules count	1 Permission entry

Inbound rules (1)

The 'Inbound rules' section shows a table with columns: Name, Security group rule ID, IP version, Type, Protocol, and Port range. The first rule is configured as follows:

Name	Security group rule ID	IP version	Type	Protocol	Port range
-	sgr-0a816827c0ffc7542	IPv4	SSH	TCP	22

Create rds-sg

1. Create security group rds-sg in prod-vpc.

- Inbound rules:
 - Type: MySQL/Aurora (3306) → Source: Custom → choose **security group** sg-bastion (this allows only instances in sg-bastion to reach the DB).
- Outbound: Allow all.

The screenshot shows the 'Create security group' page in the AWS Management Console. The 'Basic details' section is visible, with the following fields:

- Security group name:** rds-sg
- Description:** Allows SSH access to developers
- VPC:** vpc-0062e25c7bd0ebf25 (prod-vpc)

The 'Inbound rules' section is also visible, showing a rule being added:

- Type:** MySQL/Aurora
- Protocol:** TCP
- Port range:** 3306
- Source:** Custom... (selected)
- Description - optional:** (empty)

A search bar for the source security group is shown, with 'sg-0314d7087a95...' entered. A 'Delete' button is visible next to the search results.

The screenshot shows the 'Security group details' page for the security group 'sg-086723c5d22912e90 - rds-sg'. A green banner at the top states: 'Security group (sg-086723c5d22912e90 | rds-sg) was created successfully'. Below this, the 'Details' section is visible:

- Security group name:** rds-sg
- Security group ID:** sg-086723c5d22912e90
- Description:** Allows traffic to Db in pvt-subne
- VPC ID:** vpc-0062e25c7bd0ebf25
- Owner:** 383114925991
- Inbound rules count:** 1 Permission entry
- Outbound rules count:** 1 Permission entry

The 'Inbound rules' tab is selected, showing a table with one rule:

Name	Security group rule ID	IP version	Type	Protocol	Port range
-	sgr-0b34a43fa6533b77e	-	MySQL/Aurora	TCP	3306

Best practice: use SG→SG references, not IP ranges for internal traffic.

CLI (example)

```
aws ec2 create-security-group --group-name sg-bastion --description "bastion SG" --vpc-id <VPC_ID>
```

```
aws ec2 authorize-security-group-ingress --group-id <SG_BASTION_ID> --protocol tcp --port 22 --cidr <YOUR_IP/32>
```

```
aws ec2 create-security-group --group-name sg-rds --description "rds SG" --vpc-id <VPC_ID>
```

```
aws ec2 authorize-security-group-ingress --group-id <SG_RDS_ID> --protocol tcp --port 3306 --source-group <SG_BASTION_ID>
```

Optional: add CloudWatch Logs endpoint rules or NAT-related egress restrictions.

4.7 Create DB Subnet Group (required for RDS)

Console

1. RDS → Subnet groups → Create DB subnet group.
 - Name: prod-db-subnet-group.
 - VPC: prod-vpc. Add prod-private-a and prod-private-b. Save.

Create DB subnet group

To create a new subnet group, give it a name and a description, and choose an existing VPC. You will then be able to add subnets related to that VPC.

Subnet group details

Name
You won't be able to modify the name after your subnet group has been created.
prod-db-subnet-group
Must contain from 1 to 255 characters. Alphanumeric characters, spaces, hyphens, underscores, and periods are allowed.

Description
DB subnet group

VPC
Choose a VPC identifier that corresponds to the subnets you want to use for your DB subnet group. You won't be able to choose a different VPC identifier after your subnet group has been created.
prod-vpc (vpc-0062e25c7bd0ebf25)
3 Subnets, 2 Availability Zones

Add subnets

Availability Zones
Choose an availability zone
ap-south-1a X ap-south-1b X ap-south-1c X

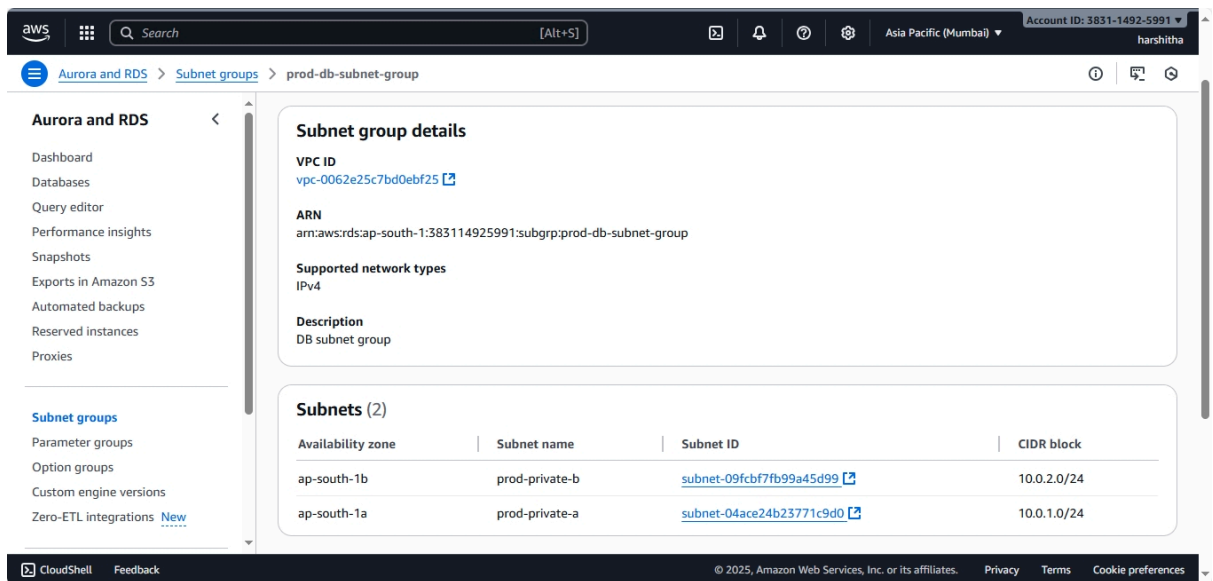
Subnets
Choose the subnets that you want to add. The list includes the subnets in the selected Availability Zones.
Select subnets
prod-private-a X prod-private-b X
Subnet ID: subnet-04ace24b23771c9d0 CIDR: 10.0.1.0/24 Subnet ID: subnet-09fcb77fb99a45d99 CIDR: 10.0.2.0/24

For Multi-AZ DB clusters, you must select 3 subnets in 3 different Availability Zones.

Subnets selected (2)

Availability zone	Subnet name	Subnet ID	CIDR block
ap-south-1a	prod-private-a	subnet-04ace24b23771c9d0	10.0.1.0/24
ap-south-1b	prod-private-b	subnet-09fcb77fb99a45d99	10.0.2.0/24

Cancel Create



CLI

```
aws rds create-db-subnet-group --db-subnet-group-name prod-db-subnet-group \
  --db-subnet-group-description "RDS subnet group for prod" \
  --subnet-ids <SUBNET_ID_A> <SUBNET_ID_B>
```

4.8 Create RDS (MySQL) — production settings

Console (recommended for first time)

1. RDS → Databases → Create database → Standard Create.
2. Engine: **MySQL**. Choose version (e.g., 8.x).
3. Templates: Production (or Free tier if testing).
4. DB instance identifier: prod-rds-db.
5. Credentials: admin + strong password (or use Secrets Manager option — recommended).
6. DB instance size: For production choose appropriate (e.g., db.m5.large), for practice use db.t3.micro.
7. Storage: General Purpose SSD (gp3 recommended for prod), size 20 GiB (adjust as needed). Enable autoscaling storage for prod.
8. Availability & durability: **Multi-AZ deployment** (recommended).

9. Connectivity:

- VPC: prod-vpc
- Subnet group: prod-db-subnet-group
- Public access: **No** (important)
- VPC security groups: select rds-sg

10. Additional configuration:

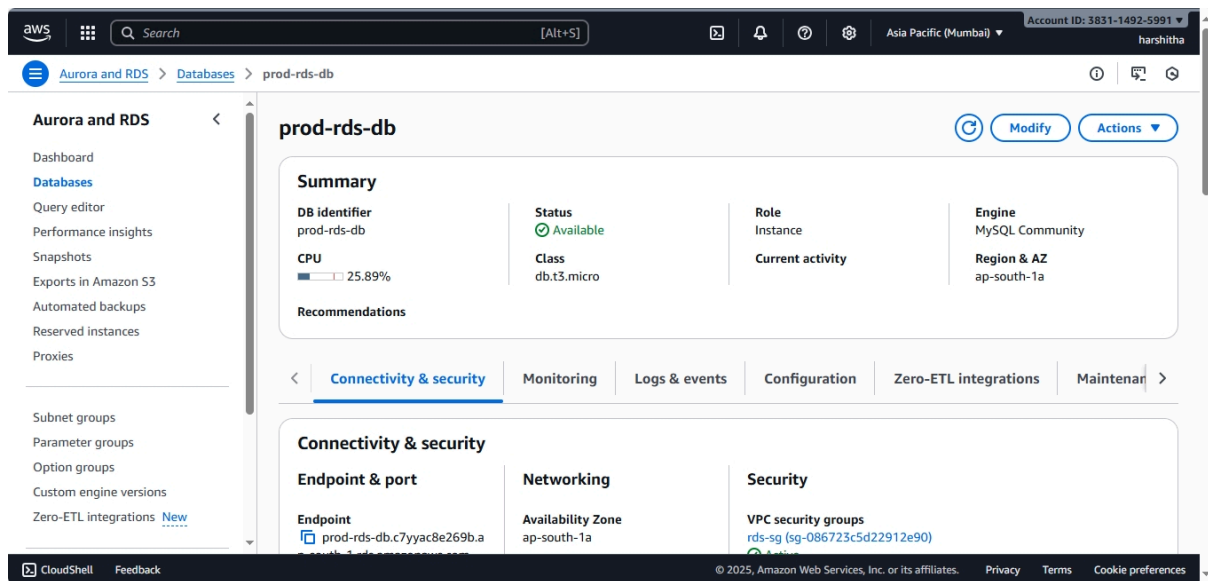
- Initial DB name: prodapp (optional)
- Backups: Enable automated backups, retention 7 days (or as required)
- Enable encryption (use AWS KMS key; default is fine for practice)
- Enable Performance Insights (recommended)
- Enhanced monitoring: enable and choose granularity (60s)
- Maintenance window: keep default or schedule

11. Create database and wait until Available.

Note: For Multi-AZ add `--multi-az` and select larger instance class for production.

The screenshot shows the AWS Management Console interface for Aurora and RDS Databases. The left sidebar contains navigation links for Aurora and RDS, including Dashboard, Databases, Query editor, Performance insights, Snapshots, Exports in Amazon S3, Automated backups, Reserved instances, Proxies, Subnet groups, Parameter groups, Option groups, Custom engine versions, and Zero-ETL integrations. The main content area displays a table of databases with the following columns: DB identifier, Status, Role, Engine, Region, and Size. The table contains three entries:

DB identifier	Status	Role	Engine	Region	Size
database-1	Available	Regional cluster	Aurora Po...	ap-south-1	2 instan
database-1-instance-1	Available	Writer instance	Aurora Po...	ap-south-1a	db.r6g.;
database-1-instance-1-ap-south-1b	Available	Reader instance	Aurora Po...	ap-south-1b	db.r6g.;



4.9 Create IAM role for EC2 to access Secrets Manager (EC2SecretsRole)

Store DB credentials in Secrets Manager and let EC2 fetch them — avoids hardcoded DB passwords.

Console: create IAM Role

1. IAM → Roles → Create role → AWS service → EC2 → Next.
2. Attach inline/custom policy (least privilege): allow `secretsmanager:GetSecretValue` for secret `prod/rds/master` and `kms:Decrypt` on KMS key if encryption is used.
3. Name role `EC2SecretsRole`. Create.

aws

Search

[Alt+S]

2

🔔

🔒

⚙️

Global

Account ID: 1831-1492-5991

harshitha

IAM

>

Roles

>

Create role

Step 1

Select trusted entity

Step 2

Add permissions

Step 3

Name, review, and create

Name, review, and create

Role details

Role name

Enter a meaningful name to identify this role.

EC2SecretsRole

Maximum 64 characters. Use alphanumeric and "+", "@", "-", "." characters.

Description

Add a short explanation for this role.

Allows EC2 instances to call AWS services on your behalf.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: "_", "+", "@", "-", "/", "(", ")", "#", "%", "^", ":", "~", "=".

Step 1: Select trusted entities

Edit

Trust policy

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": {
7         "Service": [
8           "ec2.amazonaws.com"
9         ]
10      }
11    ]
12  }
13 }
14
15
16 }
```

Step 2: Add permissions

Edit

Permissions policy summary

Policy name	Type	Attached as
ROSAKMSProviderPolicy	AWS managed	Permissions policy
SecretsManagerReadWrite	AWS managed	Permissions policy

Step 3: Add tags

Add tags - optional

Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Cancel

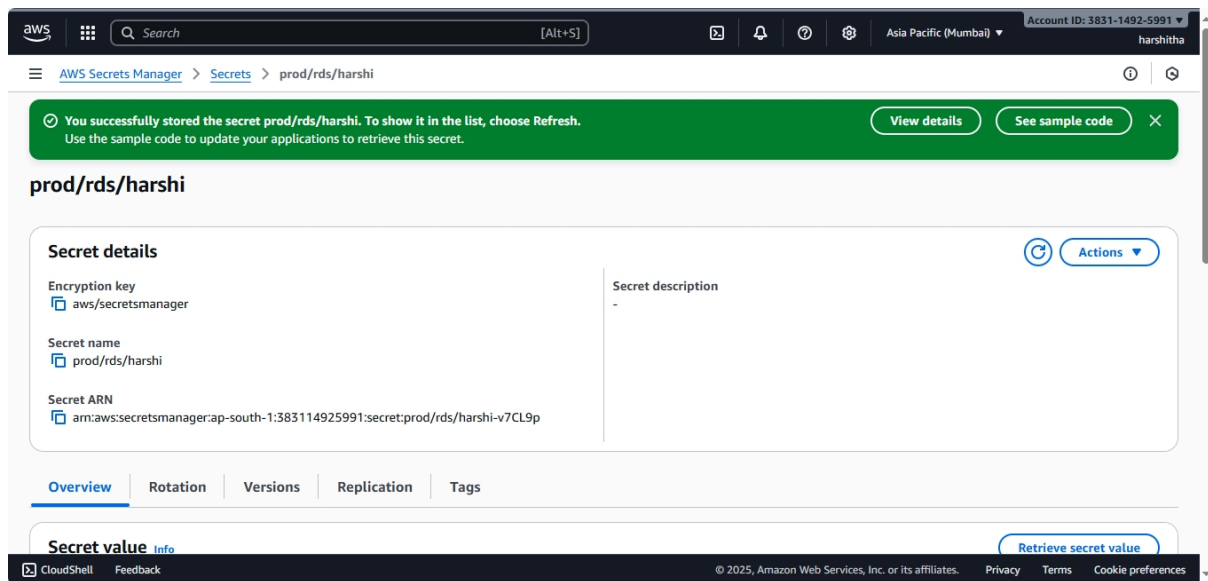
Previous

Create role

4.10 Store DB credentials in Secrets Manager

Console

1. Secrets Manager → Store a new secret → choose “Credentials for RDS database” or “Other type of secret”.
2. Name: prod/rds/harshi. Save.



CLI

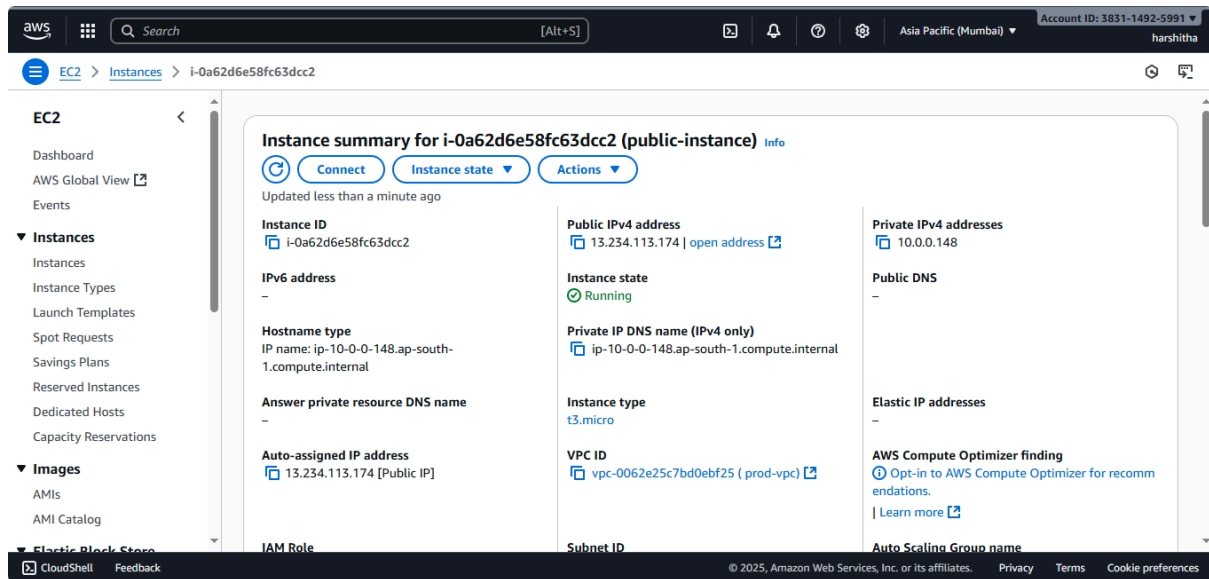
```
aws secretsmanager create-secret --name prod/rds/master \
  --description "RDS master credentials" \
  --secret-string '{"username":"admin","password":"StrongPassw0rd!"}'
```

Ensure IAM role EC2SecretsRole has permission to get this secret.

4.11 Launch EC2 bastion and attach role

Console

1. EC2 → Launch instances → choose Amazon Linux → t3.micro for testing.
2. Network: VPC prod-vpc, Subnet prod-public-a, Auto-assign public IPv4: enable.
3. Key pair: select or create
4. Security group: sg-bastion.
5. IAM role: select EC2SecretsRole.
6. Launch.



CLI

```
aws ec2 run-instances --image-id <AMI_ID> --count 1 --instance-type t3.micro \
  --key-name <KEY_NAME> --subnet-id <PUBLIC_SUBNET_ID> --iam-instance-profile
Name=EC2SecretsRole \
  --security-group-ids <SG_BASTION_ID> --associate-public-ip-address
```

Step 1: SSH into Bastion or Connect (UI)

```
ssh -i mykey.pem ec2-user@<EC2_PUBLIC_IP>
```

- Replace mykey . pem with your key pair file.
- Replace <EC2_PUBLIC_IP> with your Bastion EC2's public IP.
- Now you're inside Bastion → from here we'll connect to RDS.

Step 2: Install MySQL Client + AWS CLI + jq

On Amazon Linux 2023, the mysql package name is different. Use:

```
sudo yum update -y
```

```
sudo yum install -y mariadb105 jq
```

(If mariadb105 not found, try mariadb106)

Verify:

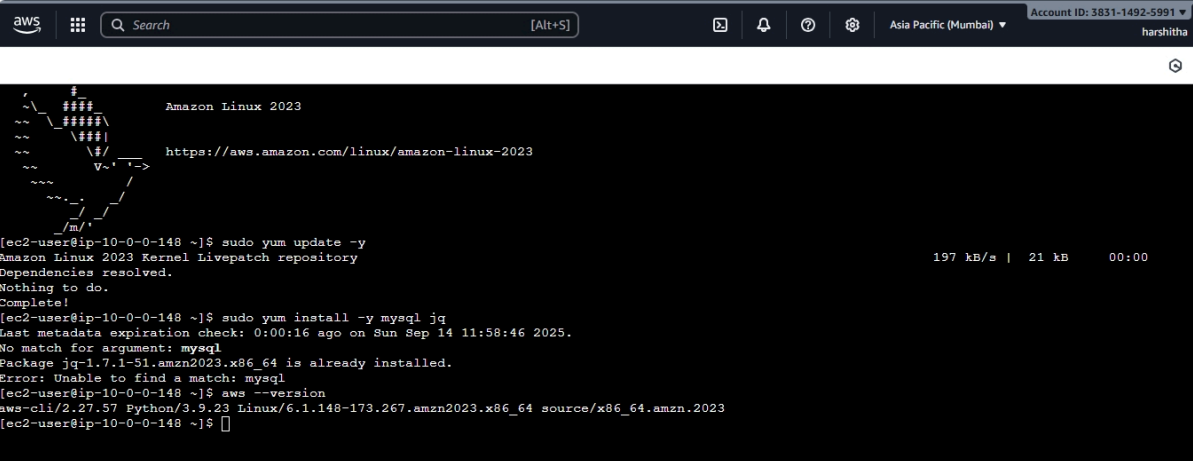
mysql --version

- mariadb → required to connect to RDS.
- jq → JSON parser, needed to extract username/password from Secrets Manager.

Check AWS CLI:

aws --version

If not available, install per [AWS CLI official docs](#).



```
aws
[Search] [Alt+S]
Account ID: 3831-1492-5991
harshitha

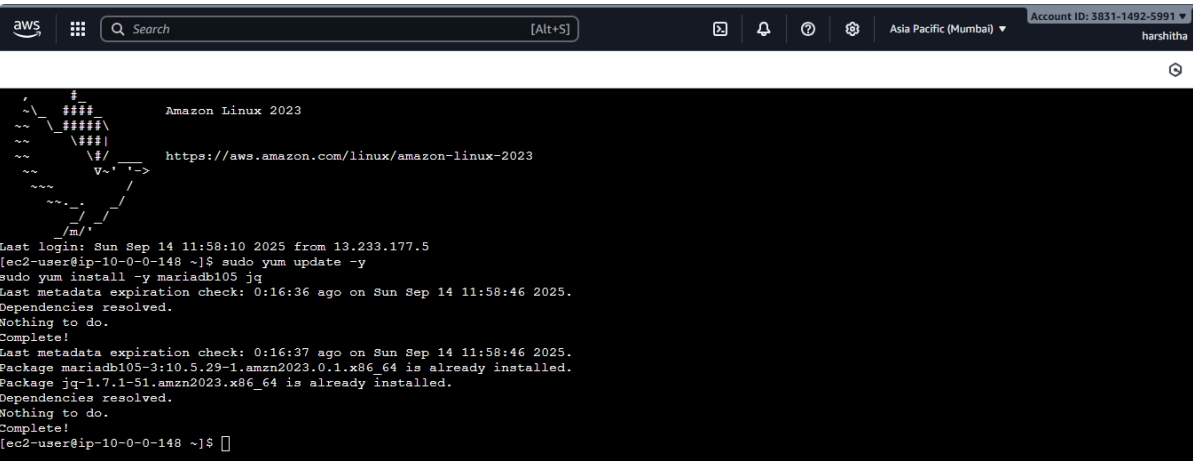
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-10-0-0-148 ~]$ sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-0-148 ~]$ sudo yum install -y mysql jq
Last metadata expiration check: 0:00:16 ago on Sun Sep 14 11:58:46 2025.
No match for argument: mysql
Package jq-1.7.1-51.amzn2023.x86_64 is already installed.
Error: Unable to find a match: mysql
[ec2-user@ip-10-0-0-148 ~]$ aws --version
aws-cli/2.27.57 Python/3.9.23 Linux/6.1.148-173.amzn2023.x86_64 source/x86_64.amzn.2023
[ec2-user@ip-10-0-0-148 ~]$
```

i-0a62d6e58fc63dcc2 (public-instance)

PublicIPs: 13.234.113.174 PrivateIPs: 10.0.0.148

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



```
aws
[Search] [Alt+S]
Account ID: 3831-1492-5991
harshitha

Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Sun Sep 14 11:58:10 2025 from 13.233.177.5
[ec2-user@ip-10-0-0-148 ~]$ sudo yum update -y
sudo yum install -y mariadb105 jq
Last metadata expiration check: 0:16:36 ago on Sun Sep 14 11:58:46 2025.
Dependencies resolved.
Nothing to do.
Complete!
Last metadata expiration check: 0:16:37 ago on Sun Sep 14 11:58:46 2025.
Package mariadb105-3:10.5.29-1.amzn2023.0.1.x86_64 is already installed.
Package jq-1.7.1-51.amzn2023.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-0-148 ~]$
```

i-0a62d6e58fc63dcc2 (public-instance)

PublicIPs: 13.234.113.174 PrivateIPs: 10.0.0.148

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Step 3: Retrieve DB credentials from Secrets Manager

Here's the secure way (instead of hardcoding passwords):

```
SECRET_JSON=$(aws secretsmanager get-secret-value \
--secret-id prod/rds/harshi\
--region ap-south-1 \
--query SecretString \
--output text)
```

```
USERNAME=$(echo $SECRET_JSON | jq -r .username)
```

```
PASSWORD=$(echo $SECRET_JSON | jq -r .password)
```

Replace `prod/rds/harshi` with your secret name.

If you're using the AWS-managed one, replace with something like:

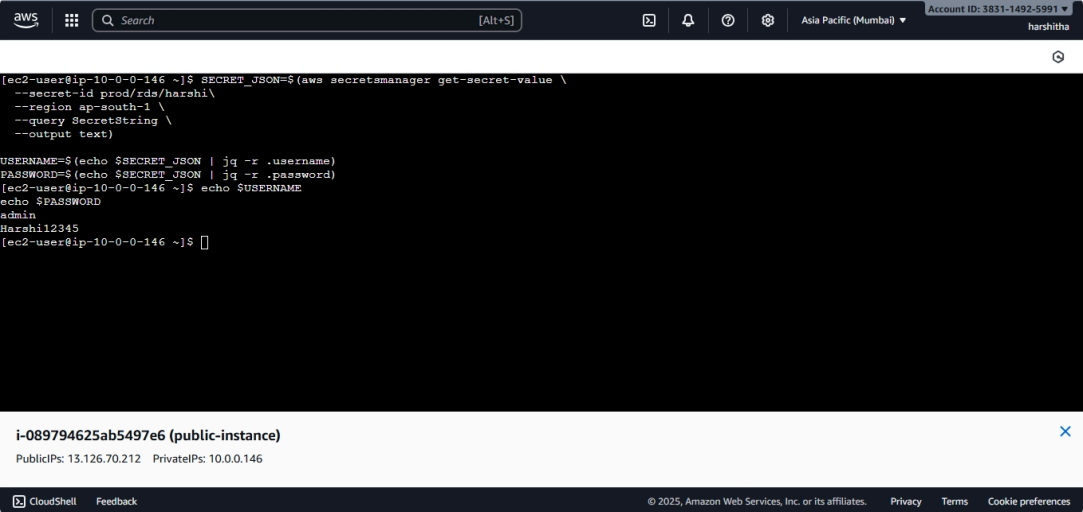
`rds!cluster-bf1f6d23-c79c-4215-aecc-70e57c63f9e7`

- Replace `ap-south-1` with your region.

Test if variables are set:

```
echo $USERNAME
```

```
echo $PASSWORD
```



The screenshot shows an AWS CloudShell terminal window. The top bar includes the AWS logo, a search bar, and account information for 'Asia Pacific (Mumbai)' with account ID '3831-1492-5991'. The terminal output shows the following commands and results:

```
[ec2-user@ip-10-0-0-146 ~]$ SECRET_JSON=$(aws secretsmanager get-secret-value \
--secret-id prod/rds/harshi\
--region ap-south-1 \
--query SecretString \
--output text)
USERNAME=$(echo $SECRET_JSON | jq -r .username)
PASSWORD=$(echo $SECRET_JSON | jq -r .password)
[ec2-user@ip-10-0-0-146 ~]$ echo $USERNAME
admin
[ec2-user@ip-10-0-0-146 ~]$ echo $PASSWORD
Harahi12345
[ec2-user@ip-10-0-0-146 ~]$
```

Below the terminal, a notification bar indicates the instance is 'i-089794625ab5497e6 (public-instance)' with public IP '13.126.70.212' and private IP '10.0.0.146'. The bottom bar shows 'CloudShell' and 'Feedback' links, along with copyright information for Amazon Web Services, Inc.

Step 4: Connect to RDS

Get RDS Endpoint from **RDS Console** → **Connectivity & security** → **Endpoint**.

RDS_ENDPOINT=prod-rds-db.c7yyac8e269b.ap-south-1.rds.amazonaws.com(replace your end point)

```
mysql -h $RDS_ENDPOINT -u $USERNAME -p"$PASSWORD"
```

If successful, you'll land in mysql> prompt. Example:

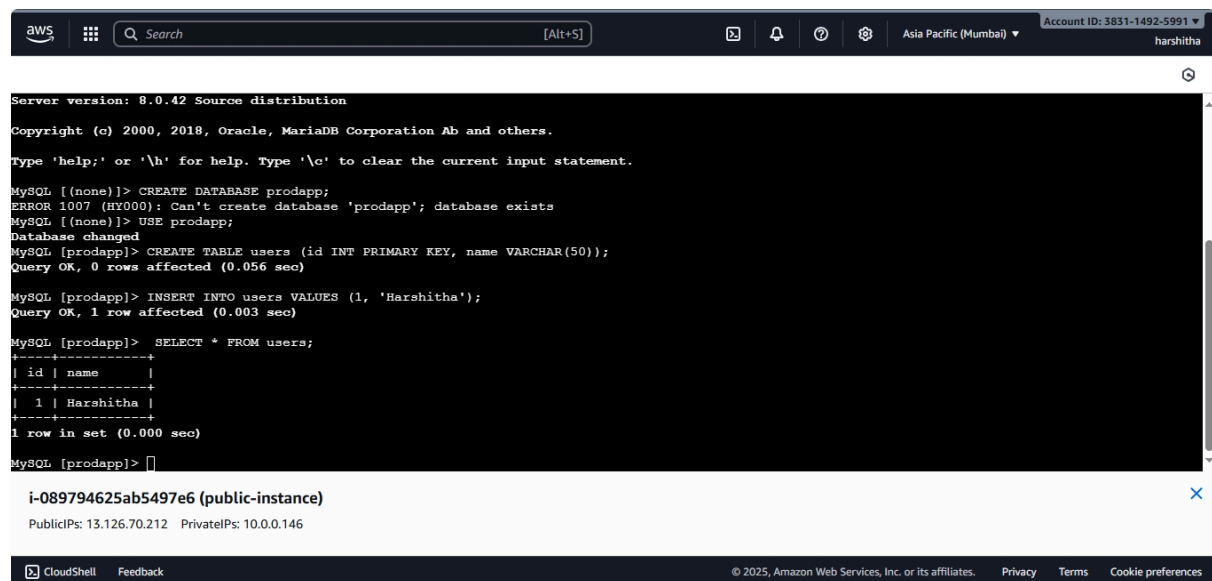
```
mysql> CREATE DATABASE prodapp;
```

```
mysql> USE prodapp;
```

```
mysql> CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(50));
```

```
mysql> INSERT INTO users VALUES (1, 'Harshitha');
```

```
mysql> SELECT * FROM users;
```



The screenshot shows an AWS CloudShell terminal window. The top bar includes the AWS logo, a search bar, and navigation icons. The terminal output shows the MySQL prompt and the following commands and results:

```
Server version: 8.0.42 Source distribution
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> CREATE DATABASE prodapp;
ERROR 1007 (HY000): Can't create database 'prodapp'; database exists
MySQL [(none)]> USE prodapp;
Database changed
MySQL [prodapp]> CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(50));
Query OK, 0 rows affected (0.056 sec)

MySQL [prodapp]> INSERT INTO users VALUES (1, 'Harshitha');
Query OK, 1 row affected (0.003 sec)

MySQL [prodapp]> SELECT * FROM users;
+----+-----+
| id | name |
+----+-----+
| 1  | Harshitha |
+----+-----+
1 row in set (0.000 sec)

MySQL [prodapp]>
```

Below the terminal output, the instance details are shown: i-089794625ab5497e6 (public-instance). Public IPs: 13.126.70.212, Private IPs: 10.0.0.146.

4.12 Optional — Export snapshot to S3 (for archiving/analytics)

If you want to export snapshot data to S3 (Parquet), create an export role and start export.

Create S3 bucket prod-rds-exports-<suffix> with encryption (SSE-KMS recommended).

Create IAM role rds-snapshot-export-role with trust for rds.amazonaws.com and policy allowing PutObject on the bucket and KMS encrypt actions. Example policy provided earlier in the conversation; attach to the role.

Start export (CLI)

```
aws rds start-export-task \  
--export-task-identifier prod-export-1 \  
--source-arn arn:aws:rds:<REGION>:<ACCOUNT_ID>:snapshot:prod-rds-snap-1 \  
--s3-bucket-name prod-rds-exports-<suffix> \  
--iam-role-arn arn:aws:iam::<ACCOUNT_ID>:role/rds-snapshot-export-role \  
--kms-key-id arn:aws:kms:<REGION>:<ACCOUNT_ID>:key/<KMS_KEY_ID>
```

Monitor with `aws rds describe-export-tasks`.

5) Tests & validation (exact commands + SQL)

5.1 Connectivity tests

- From bastion (works):
 - `mysql -h <RDS_ENDPOINT> -u admin -p` → enter password → should connect.
- From your laptop (outside VPC) — expected fail (if public access disabled). If you deliberately try, expect a connection timeout or refused.

5.2 CRUD tests (SQL)

```
CREATE DATABASE testdb;  
USE testdb;  
CREATE TABLE items (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100));  
INSERT INTO items (name) VALUES ('alpha'), ('beta');  
SELECT * FROM items;
```

5.3 Snapshot & restore test

- Console → RDS → Databases → Select prod-rds-db → Actions → Take snapshot. Name prod-rds-snap-1.
- Restore snapshot → Actions → Restore snapshot → New DB id prod-rds-restore-1.
- Connect to prod-rds-restore-1 and verify data.

5.4 Export snapshot to S3 test (if configured)

- Start export, wait completion, check S3 bucket for objects.

6) Troubleshooting — common errors & fixes

- **Cannot connect (timeout)**
 - Check RDS Publicly accessible = No (if private).
 - Confirm sg-rds inbound port 3306 allows source sg-bastion.
 - Confirm bastion is in public subnet with a public IP.
 - Verify route tables: public subnet associated with public-rt, private with private-rt.
 - If using NAT, ensure NAT is Available.
- **Access denied (authentication)**
 - Wrong username/password — verify Secrets Manager secret and that EC2 role can fetch.
 - If using IAM auth — ensure DB user exists and policy allows rds-db:connect.
- **Snapshot export fails**
 - Verify rds-snapshot-export-role has S3 PutObject, ListBucket and KMS permissions.
 - Ensure S3 bucket encryption/KMS configuration matches the role permissions.

7) Cleanup (essential to avoid charges)

Console steps for safe teardown (recommended order):

1. Stop/terminate prod-bastion EC2 instance.

2. Delete RDS instances (choose whether to keep final snapshot). For test: delete with `Skip final snapshot = Yes`.
3. Delete DB snapshots (if not needed).
4. Delete S3 export bucket contents then bucket.
5. Delete route tables, detach and delete Internet Gateway.
6. Delete subnets.
7. Delete VPC.
8. Remove IAM roles and policies (EC2SecretsRole, snapshot export role).
9. Delete Secrets Manager secrets (rotate if needed before deletion).

CLI cleanup examples

```
aws rds delete-db-instance --db-instance-identifier prod-rds-db --skip-final-snapshot
aws rds delete-db-snapshot --db-snapshot-identifier prod-rds-snap-1
aws ec2 delete-nat-gateway --nat-gateway-id <NAT_ID>
aws ec2 release-address --allocation-id <EIP_ALLOC_ID>
aws s3 rm s3://prod-rds-exports-<suffix> --recursive
aws s3 rb s3://prod-rds-exports-<suffix>
aws ec2 delete-subnet --subnet-id <SUBNET_ID>
aws ec2 detach-internet-gateway --internet-gateway-id <IGW_ID> --vpc-id <VPC_ID>
aws ec2 delete-internet-gateway --internet-gateway-id <IGW_ID>
aws ec2 delete-vpc --vpc-id <VPC_ID>
aws iam delete-role --role-name EC2SecretsRole
```